

What to run, in what order

Every command from an empty folder to a working system: what each file does, what it adds, and how to check it worked

Before anything: two rules

1. Every command runs in the project folder, with the virtual environment active.

```
cd /Volumes/Work/aws-graphrag-on-db
source .venv/bin/activate
```

Your prompt should start with `(.venv)`. **Do this in every new terminal.** Without it you get `ModuleNotFoundError: No module named 'boto3'`, which looks like a broken project and is not.

2. `python -m` is not the same as `python`.

Form	When
<code>python -m metadata.build</code>	The file imports other project files. Most of them do
<code>python scripts/rds_setup.py</code>	A standalone script

`-m` runs the file as part of the project, so `from metadata import llm` works. Running it directly gives `ModuleNotFoundError: No module named 'metadata'`.

The whole run, at a glance

#	Command	Adds	Check	Time
1	<code>pip install -r requirements.txt</code>	The packages	imports work	2 min
2	fill in <code>.env</code>	Keys and addresses	<code>git check-ignore .env</code>	5 min
3	<code>pytest tests/test_bedrock.py</code> <code>tests/test_guardrail.py -v</code>	—	8 passed	20 s
4	<code>python scripts/rds_setup.py</code>	The second database, pgvector	printed report	10 s
5	<code>pytest tests/test_rds_setup.py -v</code>	—	7 passed	5 s
6	<code>python dataplane/schema/apply_schema.py</code>	20 empty tables	printed message	20 s
7	<code>pytest tests/test_stage1_schema.py -v</code>	—	11 passed	5 s
8	<code>python -m dataplane.generator.run</code>	1,132,667 rows	printed counts	1-5 min
9	<code>pytest tests/test_stage2_data.py -v</code>	—	Gate 1: 13 passed	10 s
10	<code>pytest tests/test_neptune.py -v</code>	—	4 passed	10 s
11	<code>python -m metadata.build</code>	Links found, notes drafted	3 undeclared links	1-3 min
12	<code>python -m metadata.review</code>	A person reads it	the notes and links	5 min
13	<code>python -m metadata.review --approve-all</code>	Approval	20 notes, 3 links	1 s
14	<code>python -m metadata.publish</code>	1,415 embeddings, the graph	20 nodes, 22 edges	1-2 min
15	<code>pytest tests/test_stage5_metadata.py -v</code>	—	Gate 2: 19 passed	5 s
16	<code>pytest tests/test_stage6_agent.py -v</code>	—	18 passed	1-3 min
17	<code>uvicorn api.main:app --port 8000</code>	The website	the demo page	—
18	<code>python -m evals.run_eval</code>	A scorecard	18 questions scored	1-4 min

Steps 4 to 16 run once. After that, day to day, you only need steps 17 and 18.

Step 1: Install the packages

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

What it adds: boto3 for AWS, psycopgp for PostgreSQL, requests for Neptune, LangGraph for the agent, FastAPI for the website, sqlglot for the SQL checks, polars and numpy for generating data, pytest for the tests.

Check:

```
python -c "import boto3, psycopgp, requests, langgraph, fastapi, sqlglot; print('ok')"
```

Step 2: Fill in `.env`

```
cp .env.example .env
```

Setting	Where it comes from	Used by
AWS_ACCESS_KEY , AWS_SECRET_KEY	aws iam create-access-key	Every AWS call
AWS_MODEL_BEDROCK_ID	aws bedrock list-inference-profiles	The chat model
AWS_BEDROCK_ARN	The same listing	The region is read from this
BEDROCK_GUARDRAIL_ID , BEDROCK_VERSION	aws bedrock create-guardrail	The prompt-attack check
AWS_REGION	Must match the ARN	Tracing
DATA_DB_URL , AGENT_DB_URL	The RDS endpoint, your username and password	Both databases
NEPTUNE_CLUSTER_HOST , NEPTUNE_PORT	aws neptune describe-db-clusters	The graph

Check that it can never be committed:

```
git check-ignore .env # must print: .env
```

If it prints nothing, stop: your keys are one `git add .` away from being public.

Step 3: Prove the AWS side works

```
pytest tests/test_bedrock.py tests/test_guardrail.py -v
```

What it checks: every `.env` setting is filled in, `AWS_REGION` matches the ARN's region, the chat model replies with valid JSON for both roles, the embedding model returns exactly 1,024 numbers, the guardrail is reachable, allows a normal question, and blocks an attack.

Expect: 8 passed. Do this before anything else, because a wrong key or region here would otherwise show up as a confusing failure three steps later.

Failure	Cause
Missing in .env: ...	That setting is blank
UnrecognizedClientException	The key pair is wrong
on-demand throughput isn't supported	A plain model ID was used instead of the inference profile ID
INVALID_PAYMENT_INSTRUMENT	The account has no payment method AWS Marketplace accepts

Step 4: Prepare the database

```
python scripts/rds_setup.py
```

What the file does: connects to the RDS instance, creates the **second** database `agent_meta`, and switches on **pgvector** inside it.

What it adds to the service: RDS gives you one database. This project needs two: `mf_data` for the fund data, which the agent may only read, and `agent_meta` for the agent's own notes, links and embeddings, which it may write. Keeping them apart means the agent's write access can never reach fund data.

Expected output:

```
RDS instance: fundgraph-db.xxxx.ap-south-1.rds.amazonaws.com
fund data : mf_data
agent data : agent_meta
Connected. Server: PostgreSQL 17.11 on aarch64-unknown-linux-gnu
Database 'agent_meta': created.
Extension 'vector' in agent_meta: version 0.8.2
mf_data: 0 table(s) in the public schema
agent_meta: 0 table(s) in the public schema
```

Safe to run again: an existing `agent_meta` is left alone.

Step 5: Check the databases

```
pytest tests/test_rds_setup.py -v
```

What it checks: both databases answer, they are on the **same instance** (different hosts would mean you are paying for two), pgvector is not just listed but actually works, PostgreSQL is version 15 or newer, and Apache AGE is absent, which is expected on RDS.

Expect: 7 passed.

Failure	Cause and fix
ConnectionTimeout	Your public IP changed, or the instance is stopped. Run <code>./scripts/allow_my_ip.sh</code> , or start the instance
password authentication failed	The password in <code>.env</code> does not match RDS

Step 6: Create the 20 tables

```
python dataplane/schema/apply_schema.py
```

What the file does: runs four SQL files in order against `mf_data`.

File	Creates
01_fund_house.sql	9 tables: categories, benchmarks, schemes, plans, managers, daily prices, monthly AUM
02_market.sql	6 tables: sectors, groups, issuers, securities, credit ratings, holdings
03_investors.sql	4 tables: distributors, investors, folios, SIPs
04_transactions.sql	1 table, split into 60 monthly pieces

What it adds to the service: the shape of the fund database, including the awkward parts that make it realistic: **three links that are never declared**, and **two tables with cryptic old-style column names** such as `rtg_dt` and `arn_no`.

The order matters. Later files point at tables created by earlier ones, which is why they are numbered. Everything happens in one transaction, so a failure leaves you with no half-built schema.

Expected output: four `Running ...` lines, then `Schema applied: all 20 tables created in mf_data`.

Safe to re-run: each file drops its own tables first. That also means **every run deletes the data**, which is fine here because the data is generated in minutes. Never point this at a database holding real data.

Step 7: Check the schema

```
pytest tests/test_stage1_schema.py -v
```

What it checks: all 20 tables exist, `transactions` really has its 60 monthly pieces, the three hidden links are **still undeclared** (if a foreign key crept in, the discovery step would have nothing to discover), the 19 declared links are present, and the `transactions` columns are exactly as the generator expects.

Expect: 11 passed.

Step 8: Fill the database

```
python -m dataplane.generator.run
```

What the file does: runs the generator files in order, then hands everything to the loader.

File	Builds
<code>common.py</code>	Settings, trading days, month ends, name and city lists
<code>patterns.py</code>	The six planted stories as fixed values
<code>fund_house.py</code>	Tables 1-8: schemes, plans, managers, five years of daily prices
<code>market.py</code>	Tables 10-14: issuers, securities, credit ratings
<code>investors.py</code>	Tables 16-19: investors, accounts, SIPs
<code>transactions.py</code>	Table 20: every transaction
<code>summaries.py</code>	Tables 9 and 15, calculated from the transactions
<code>load/copy_loader.py</code>	Sends all 20 tables to RDS with PostgreSQL's fast COPY

What it adds to the service: 1,132,667 rows of realistic fund data, and the six stories that give demo questions real answers. **Code creates every number; no model creates any data.** The same seed produces identical data every run.

Expected output: steps `1/5` to `5/5`, then twenty `loading ...` lines, ending with `Done: 1,132,667 rows across 20 tables in 53s`.

Allow 1 to 5 minutes: about 140 MB travels to Mumbai and is loaded on a small instance.

Step 9: Gate 1

```
pytest tests/test_stage2_data.py -v
```

What it checks: no broken links, $\text{units} \times \text{NAV}$ equals the amount on every transaction, no account holds fewer than zero units, holding weights add to 100 per scheme per month, **both routes to AUM agree exactly**, and all six planted stories are visible in the data.

Expect: 13 passed. This is a gate: do not continue until it passes. Everything later is built on this data being correct.

Step 10: Check the graph database

```
pytest tests/test_neptune.py -v
```

What it checks: the cluster answers a query at all, which proves three separate things at once: your IP is allowed on port 8182, the request signing works, and the `neptune-db` permissions are in place. It then writes a node, reads it back, and deletes it.

Expect: 4 passed.

Failure	Cause
Timeout	The cluster is stopped, or your IP changed (<code>./scripts/allow_my_ip.sh</code>)
403 or not authorized	The <code>neptune-db</code> permissions are missing, or IAM authentication is off on the cluster

Step 11: Learn the database

```
python -m metadata.build
```

What the file does, in four stages:

1. **Reads the catalog** through `adapters/postgres.py`: tables, columns, types, keys, and statistics such as how many different values each column holds.
2. **Finds links** with `links.py`, three ways: declared foreign keys; columns with the same name whose values match; and columns with **different** names

whose values match.

3. **Drafts notes** with `notes.py` and `llm.py`: the chat model writes a plain-English description of every table and column, four tables at a time.

4. **Saves** everything to `agent_meta` as **unapproved**.

What it adds to the service: this is what lets the system work on a database it has never seen. The three hidden links are found here, by checking values rather than by being told.

Expected output:

```
Reading catalog and statistics from the fund database ...
  20 tables, 19 declared foreign keys
Finding links between tables ...
  19 declared, 3 undeclared links proposed
Drafting notes for 20 of 20 tables (0 unchanged since the last run) ...
  drafted benchmark_values
  ... (20 lines)
Done. Next: python -m metadata.review
```

Only changed tables are redrafted. Each table gets a fingerprint of its columns and types, so a second run costs almost nothing. To force a redraft:

```
python -m metadata.build --redraft.
```

Step 12: Read what it wrote

```
python -m metadata.review
```

This is the one step a person must actually do. It prints every note and every proposed link. Read them:

- Is `rtg_dt` described as the date a rating was given?
- Are the three proposed links `sec_cd → security_code`, `scheme_id → scheme_id`, and `arn_code → arn_no`?
- Does any note describe a column wrongly?

To reject a wrong link:

```
python -m metadata.review --reject-link 21
```

Why it matters: everything the SQL-writing model knows about your database comes from these notes. An approval step means a wrong note is caught by a person, not discovered later in an answer given to a client.

Step 13: Approve

```
python -m metadata.review --approve-all
```

Expected output: Approved 20 table notes and 3 links.

Step 14: Publish

```
python -m metadata.publish
```

What the file does: takes **only approved** items, sends each note and each category value to Titan to be turned into 1,024 numbers, stores them in `pgvector`, then wipes and rewrites the graph in `Neptune` from the approved links.

What it adds to the service: two lookups the agent depends on. **pgvector** lets a question find the right tables, and a loosely typed name find its exact value. **Neptune** holds how the tables connect, so the model never has to guess a join.

Expected output:

```
Embedding 1,415 items (20 table notes, 1395 values) ...
Neptune graph: 20 table nodes, 22 join edges.
Done. Next: pytest tests/test_stage5_metadata.py -v
```

`meta_edges` in `agent_meta` stays the source of truth; the graph is a copy, rebuilt each time, so a rejected link cannot survive in it.

Step 15: Gate 2

```
pytest tests/test_stage5_metadata.py -v
```

What it checks: every table has an approved note, every column has a description, personal-data columns are flagged, **at least 2 of the 3 hidden links were found**, no wrong link was approved, no personal data was embedded, the graph matches the approved links exactly, and a question finds the right tables.

Expect: 19 passed. The second gate.

The hidden-link test scores against `evals/answer_key/hidden_links.yaml`, a file the pipeline never reads. That is what makes the score meaningful.

Step 16: Test the agent

```
pytest tests/test_stage6_agent.py -v
```

What it checks: nine offline SQL-safety checks (multiple statements refused, writes refused, unknown tables refused, a missing date range on the big table refused), then every demo question through the whole agent, then that attacks and data-change requests are blocked.

Expect: 18 passed, in 1 to 3 minutes.

For only the fast offline checks:

```
pytest tests/test_stage6_agent.py -v -k checks
```

Step 17: Run it

```
uvicorn api.main:app --port 8000
```

Open	What
http://localhost:8000	The demo page: the 11 steps, the graph, the answer, the SQL and its checks
http://localhost:8000/graph	The clickable schema graph
http://localhost:8000/docs	The API, to try <code>POST /query</code> by hand

Keep this terminal open. Errors appear here. `Ctrl+C` stops the server.

After re-running `metadata.publish`, restart the server: the notes and the graph edges are read once at startup.

Step 18: Score the answers

```
python -m evals.run_eval
```

What the file does: asks 18 questions with known correct answers, scores each one, prints a scorecard and saves the run to `evals/results/`.

Number	Means
recall@5	Of the tables needed, how many it found. The one to watch: a missing table means the question cannot be answered
precision@5	Of the tables it chose, how many were needed. Spare tables are untidy, not fatal
join path	How many questions used every required join, including the hidden links
grounded	How many explanations quoted only figures that appear in the rows
complete	How many mentioned the fund, period or metric asked about
REFUSING	Whether the three questions that must be refused were refused

Other ways to run it:

```
python -m evals.run_eval --only p1      # one question, about 15 seconds
python -m evals.run_eval --judge        # plus a model's opinion of the wording
python -m evals.run_eval --ragas        # plus RAGAS, if installed
```

When to run it: after changing a prompt, or after switching the model in `.env`. Not after every save: it costs about 70 Bedrock calls.

How to read a change: with 18 questions, one question flipping moves a score by about 0.06. Only a change of several questions means something real.

Day to day, after the first build

```
cd /Volumes/Work/aws-graphrag-on-db
source .venv/bin/activate

# if the databases were stopped overnight (run in AWS CloudShell)
# aws rds start-db-instance --region ap-south-1 --db-instance-identifier fundgraph-db
# aws neptune start-db-cluster --region ap-south-1 --db-cluster-identifier fundgraph-graph

./scripts/allow_my_ip.sh          # if your internet address changed
pytest tests/test_rds_setup.py tests/test_neptune.py -q    # 11 passed
uvicorn api.main:app --port 8000
```

At the end of the day, to stop paying for idle databases:

```
aws rds stop-db-instance --region ap-south-1 --db-instance-identifier fundgraph-db
aws neptune stop-db-cluster --region ap-south-1 --db-cluster-identifier fundgraph-graph
```

Both restart by themselves after 7 days.

Which test to run after which change

You changed	Run
Any code	<code>pytest tests/ -q --all 80</code>
A schema file	Steps 6 to 9
A generator file	Steps 8 and 9
A prompt	<code>pytest tests/test_stage6_agent.py -v</code> , then <code>python -m evals.run_eval</code>
The model in <code>.env</code>	<code>pytest tests/test_bedrock.py -v</code> , then the full evaluation, and compare scorecards
<code>config/metrics.yaml</code> or the notes	Steps 14 to 16, then restart the server
Nothing, but it stopped working	<code>pytest tests/test_rds_setup.py tests/test_neptune.py -q</code> first: it is usually the IP or a stopped database

Starting completely fresh

```
python dataplane/schema/apply_schema.py # 20 empty tables again
python -m dataplane.generator.run # the same data, from the same seed
python -m metadata.build --redraft # redraft every note
python -m metadata.review --approve-all
python -m metadata.publish
pytest tests/ -q
```

The seed means the regenerated data is identical, so the planted stories and every expected answer still hold.

When something fails

Message	Meaning	Fix
ModuleNotFoundError: No module named 'boto3'	The virtual environment is not active	<code>source .venv/bin/activate</code>
ModuleNotFoundError: No module named 'metadata'	Run from the wrong folder, or <code>-m</code> left out	<code>cd</code> to the project; use <code>python -m ...</code>
ConnectionTimeout	IP changed, or a database is stopped	<code>./scripts/allow_my_ip.sh</code> , or start it
password authentication failed	<code>.env</code> does not match the database password	Update both URLs in <code>.env</code>
Missing in .env: ...	A blank setting	Fill it in
No JSON object in model reply	The model answered in prose	Re-run; if it repeats, raise <code>max_tokens</code>
expected 1536 dimensions, not 1024	Embeddings table left over from another build	<code>DROP TABLE meta_embeddings;</code> in <code>agent_meta</code> , then steps 11 to 15
New notes not being used	They are read once when the server starts	Restart the server
A test that passed yesterday fails today	Almost always the IP or a stopped database	Run step 5 and step 10 first